

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Reporte #6 - Semana 7 – Libro C - Capítulo 2
Tipos, operadores y expresiones

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 09/09/2024

Bibliografía:

B. W. Kernighan and D. M. Ritchie, *The C programming language*, 2nd ed. Pearson, 2002, pp. 35–49.

Abstract— *This document outlines core C programming concepts, including variable naming conventions, data types, constants, and operators. It describes the rules for naming variables, different data types and their sizes, and the usage of constants. Key operators such as arithmetic, bitwise, and conditional operators are explained. The document includes a practical C code example demonstrating these concepts through variable declarations, arithmetic operations, bitwise manipulations, and conditional logic.*

I. Nombres de variables

Los nombres de variables en C deben comenzar con una letra y pueden incluir dígitos y guiones bajos (_). No deben empezar con un guión bajo, ya que estos nombres a menudo están reservados para las rutinas de la biblioteca. Las letras mayúsculas y minúsculas se distinguen (por ejemplo, x y X son diferentes). Los nombres de variables deben ser descriptivos y evitar confusiones tipográficas. Para variables locales, se tiende a usar nombres cortos, mientras que para variables externas, se utilizan nombres más largos.

II. Tipos de datos y tamaños

Los tipos básicos en C incluyen:

- **char:** un byte, almacena un carácter.
- **int:** entero, tamaño natural en la máquina.
- **float:** punto flotante de precisión simple.
- **double:** punto flotante de doble precisión.

Se pueden aplicar calificadores como **short** y **long** a los enteros. **short** es típicamente de 16 bits y **long** de al menos 32 bits. Los enteros pueden ser **signed** (con signo) o **unsigned** (sin signo). Los **unsigned** son siempre positivos o cero y siguen la aritmética módulo 2^n . El tipo **long double** se usa para precisión extendida en flotantes, con tamaños definidos por la implementación.

III. Constantes

- **Constantes enteras:** Ejemplo, 1234 (int). Las constantes largas terminan con 'L' (por ejemplo, 123456789L), y las constantes sin signo terminan con 'U' o 'UL'.
- **Constantes en punto flotante:** Contienen un punto decimal o exponente. Por defecto, son del tipo double. El sufijo 'f' indica float y 'l' indica long double.
- **Constantes en octal y hexadecimal:** Se representan con prefijos 0 (octal) o 0x (hexadecimal). Ejemplo, 31 decimal es 037 octal o 0x1F hexadecimal.
- **Constantes de carácter:** Representadas entre comillas simples, como 'x'. Su valor es el valor numérico del carácter en el conjunto de caracteres de la máquina (por ejemplo, '0' tiene el valor 48 en ASCII).
- **Secuencias de escape:** Representan caracteres especiales, como '\n' (nueva línea) y '\xhh' (hexadecimal).

Expresiones Constantes

Una expresión constante utiliza solo constantes y puede ser evaluada durante la compilación. Se puede usar en cualquier lugar donde se necesiten constantes, como en las directivas **#define** para definir valores constantes o tamaños de arreglos.

Constantes de Enumeración

Una enumeración es una lista de valores enteros constantes, por ejemplo:

```
enum boolean { NO, YES };
enum escapes { BELL = '\a', BACKSPACE = '\b', TAB = '\t',
NEWLINE = '\n' };
```

El primer valor en una enumeración es 0, el siguiente 1, y así sucesivamente, a menos que se especifiquen valores explícitos. Los nombres en diferentes enumeraciones deben ser distintos, pero los valores dentro de una misma enumeración no necesitan ser únicos.

Las enumeraciones asocian valores constantes con nombres y son una alternativa a las directivas **#define**, con la ventaja de que los valores pueden ser generados automáticamente. Los compiladores no verifican los valores almacenados en variables de tipo enumeración, pero ofrecen una forma más manejable y depurable de trabajar con constantes.

IV. Declaraciones

En C, todas las variables deben ser declaradas antes de su uso. Una declaración especifica un tipo y contiene una lista de una o más variables de ese tipo. Las declaraciones pueden realizarse de manera implícita por el contenido, pero es común declarar las variables explícitamente como:

```
int lower, upper, step;
char c, line[1000];
```

También es posible declarar variables en líneas separadas:

```
int lower;
int upper;
int step;
char c;
char line[1000];
```

Las variables pueden ser inicializadas en el momento de la declaración. Si el nombre de la variable está seguido por un signo igual y una expresión, la expresión sirve como inicializador:

```
char esc = '\\';
int i = 0;
int limit = MAXLINE+1;
float eps = 1.0e-5;
```

Para variables no automáticas, la inicialización se realiza una vez, antes de que el programa comience a ejecutarse, y el inicializador debe ser una expresión constante. Las variables automáticas se inicializan cada vez que se entra en la función o bloque al que pertenecen, y el inicializador puede ser cualquier expresión. Las variables externas y estáticas se inicializan a cero por defecto. Las variables automáticas sin inicializador explícito tienen valores indefinidos.

El calificador `const` se puede usar para declarar variables cuyo valor no cambiará:

```
const double e = 2.71828182845905;
const char msg[] = "warning: ";
```

También se puede usar `const` con argumentos de arrays para indicar que la función no modificará ese array:

```
int strlen(const char[]);
```

V. Operadores aritméticos

Los operadores aritméticos binarios son `+`, `-`, `*`, `/` y el operador de módulo `%`. La división de enteros trunca cualquier parte fraccionaria. El operador `%` produce el residuo de la división de `x` entre `y` y es cero si `y` divide a `x` exactamente. Los operadores `+` y `-` tienen la misma precedencia, que es menor que la de `*`, `/` y `%`. Los operadores aritméticos se asocian de izquierda a derecha.

VI. Operadores relacionales y lógicos

Los operadores relacionales son `>`, `>=`, `<`, `<=`, y tienen la misma precedencia. Los operadores de igualdad `==` y `!=` tienen menor precedencia que los relacionales. Los operadores lógicos `&&` y `||` se evalúan de izquierda a derecha y la evaluación se detiene tan pronto como se conoce la verdad del resultado. El operador `&&` tiene mayor precedencia que `||`, y ambos tienen menor precedencia que los operadores relacionales y de igualdad. El operador unario `!` invierte el valor lógico de su operando, convirtiendo cualquier valor distinto de cero en 0 y un operando cero en 1.

VII. Conversiones de tipos

Cuando un operador tiene operandos de diferentes tipos, estos se convierten a un tipo común según ciertas reglas. Las conversiones automáticas convierten un operando "más estrecho" a uno "más amplio" sin perder información, como convertir un entero a punto flotante. Los caracteres (`char`) se pueden usar libremente en expresiones aritméticas.

Las conversiones implícitas siguen estas reglas generales:

- Si alguno de los operandos es `long double`, el otro se convierte a `long double`.
- De lo contrario, si alguno es `double`, el otro se convierte a `double`.
- De lo contrario, si alguno es `float`, el otro se convierte a `float`.
- De lo contrario, `char` y `short` se convierten a `int`.
- Luego, si alguno es `long`, el otro se convierte a `long`.

Las conversiones explícitas se pueden forzar mediante un operador de conversión (`cast`). Por ejemplo, para convertir un entero a `double` antes de pasarlo a una función que espera un `double`, se utiliza:

```
sqrt((double) n);
```

Esto convierte el valor de `n` a `double` sin alterar el valor de `n` en sí. La conversión en función de argumentos también ocurre según el prototipo de la función, y las conversiones automáticas se realizan si no se proporciona un prototipo.

VIII. Operadores de incremento y decremento

C proporciona dos operadores inusuales para incrementar y decrementar variables: el operador de incremento `++` y el operador de decremento `--`. El operador `++` suma 1 a su operando, mientras que el operador `--` resta 1. Estos operadores pueden usarse como prefix o postfix. Como prefix (`++n`), incrementa el valor antes de usarlo; como postfix (`n++`), incrementa el valor después de usarlo. Por ejemplo, en `x = n++`, `x` se asigna el valor original de `n` antes de que `n` se incremente. En cambio, en `x =`

`++n`, `x` recibe el nuevo valor incrementado de `n`. Estos operadores sólo se aplican a variables y no a expresiones, como `(i+j)++`.

IX. Operadores Bitwise

C ofrece seis operadores para manipulación de bits, aplicables únicamente a operandos integrales (`char`, `short`, `int`, y `long`):

- `&`: AND a nivel de bits
- `|`: OR inclusivo a nivel de bits
- `^`: XOR exclusivo a nivel de bits
- `<<`: Desplazamiento a la izquierda
- `>>`: Desplazamiento a la derecha
- `~`: Complemento a uno (unario)

El operador AND (`&`) enmascara bits específicos; por ejemplo, `n = n & 0177` pone a cero todos los bits menos los 7 bits de menor orden de `n`. El OR (`|`) activa bits; por ejemplo, `x = x | SET_ON` enciende los bits que están en `SET_ON`. El XOR (`^`) pone un 1 en los bits donde los operandos tienen bits diferentes. Los operadores de desplazamiento (`<<` y `>>`) mueven los bits a la izquierda o a la derecha, respectivamente. El operador unario `~` invierte todos los bits de su operando.

X. Operadores de asignación y expresiones

Los operadores de asignación permiten realizar operaciones y asignar el resultado a una variable en una sola expresión. Por ejemplo, `i += 2` es equivalente a `i = i + 2`, pero de forma más concisa. La mayoría de los operadores binarios tienen un operador de asignación correspondiente (`+=`, `-=`, `*=`, `/=`, `%=`). Estos operadores simplifican el código y pueden ayudar a los compiladores a generar código más eficiente. Además, los operadores de asignación pueden usarse en expresiones, como en `while ((c = getchar()) != EOF)`.

Los operadores de asignación también tienen un tipo de valor que corresponde al tipo del operando izquierdo, y el valor es el resultado después de la asignación.

XI. Expresiones condicionales

Expresión Condicional (?:): La expresión condicional se usa para simplificar construcciones `if-else`. En la forma `expr1 ? expr2 : expr3`, se evalúa `expr1`; si es verdadera, se evalúa y se devuelve `expr2`. Si es falsa, se evalúa y se devuelve `expr3`. Por ejemplo:

```
z = (a > b) ? a : b;
```

Este código asigna a `z` el valor máximo entre `a` y `b`. La expresión condicional se puede utilizar en cualquier lugar donde se permita una expresión.

Ejemplo de Uso:

```
for (i = 0; i < n; i++)
    printf("%6d%c", a[i], (i % 10 == 9 || i == n - 1) ? '\n' :
    ' ');
```

Aquí se imprime un salto de línea cada diez elementos o al final del array, de manera compacta en comparación con un if-else equivalente.

XII. Precedencia y orden de evaluación

Tabla de Precedencia y Asociatividad: Los operadores tienen distintos niveles de precedencia que determinan el orden en el que se evalúan. Los operadores con la misma fila tienen la misma precedencia. Los operadores se evalúan de acuerdo a su asociatividad, que puede ser de izquierda a derecha o de derecha a izquierda.

Operators	Associativity
() [] -> .	left to right
! ~ ++ -- + - * (type) sizeof	right to left
* / %	left to right
+ -	left to right
<< >>	left to right
< <= > >=	left to right
== !=	left to right
&	left to right
^	left to right
	left to right
&&	left to right
	left to right
?:	right to left
= += -= *= /= %= &= ^= = <<= >>=	right to left
,	left to right

Figura 1. Precedencia y asociatividad de los operadores

Orden de Evaluación: El lenguaje C no especifica el orden en que se evalúan los operandos de un operador, excepto para ciertos operadores (&&, ||, ?:, y ,). Por ejemplo, en la expresión `x = f() + g();`, las funciones `f` y `g` pueden evaluarse en cualquier orden, lo que puede afectar el resultado si alteran variables compartidas. Para evitar dependencias en el orden de evaluación, se recomienda almacenar resultados intermedios en variables temporales.

Ejemplo problemático:

```
a[i] = i++;
```

Este código puede ser interpretado de diferentes maneras por diferentes compiladores, ya que el subíndice puede tomar el valor antiguo o el nuevo de i. Dependencias en el orden de evaluación pueden llevar a comportamientos no definidos y se deben evitar en la práctica de programación.

XIII. Ejemplo práctico

Para ilustrar los conceptos discutidos, se presenta a continuación un código en C que abarca nombres de variables, tipos de datos, constantes, operadores aritméticos, operadores de incremento y decremento, operadores bitwise, y la expresión condicional.

```
#include <stdio.h>

#define MAX 100 // Constante para tamaño máximo

// Enumeración para días de la semana
enum days { SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY };

int main() {
    // Declaraciones e inicialización
    int count = 0;
    float average = 0.0;
    const char newline = '\n';
    unsigned int flags = 0xF0; // Constante hexadecimal

    // Variables para operadores bitwise
    unsigned int mask = 0x0F;
    unsigned int result;

    // Variables de enumeración
    enum days today = WEDNESDAY;

    // Operadores aritméticos
    int sum = 5 + 3;           // Suma
    int difference = 10 - 4;   // Resta
    int product = 7 * 2;       // Multiplicación
    float quotient = 10.0 / 4; // División
    int remainder = 10 % 3;    // Módulo

    // Operadores de incremento y decremento
    count++; // Incremento postfix
    ++count; // Incremento prefix

    // Operadores bitwise
    result = flags & mask; // AND a nivel de bits
    result |= mask;        // OR inclusivo a nivel de bits
```

```

result ^= mask;           // XOR exclusivo a nivel de bits
result <<= 2;             // Desplazamiento a la izquierda
result >>= 1;             // Desplazamiento a la derecha

// Expresión condicional
char dayType = (today == SATURDAY || today == SUNDAY) ? 'W' :
'B'; // W para weekend, B para business day

// Imprimir resultados
printf("Sum: %d\n", sum);
printf("Difference: %d\n", difference);
printf("Product: %d\n", product);
printf("Quotient: %.2f\n", quotient);
printf("Remainder: %d\n", remainder);
printf("Count: %d\n", count);
printf("Flags result after bitwise operations: 0x%X\n", result);
printf("Day type: %c\n", dayType);

// Imprimir un salto de línea
printf("This is a new line%c", newline);

return 0;
}

```

Explicación:

1. **Nombres de variables:** en el código, las variables count, average, newline, flags, y result están declaradas con nombres descriptivos. Count y average son nombres cortos utilizados para variables locales, mientras que flags es una variable con un nombre más significativo, adecuada para operaciones bitwise.
2. **Tipos de datos y tamaños:** se utilizan los tipos de datos int y unsigned int para las variables que manejan enteros, float para la variable average, y char para newline. Estos tipos se eligen según las necesidades de almacenamiento y precisión.
3. **Constantes:** el código muestra el uso de constantes como max, definida mediante #define, y newline, declarada con const char. También se incluye el uso de constantes hexadecimales en las variables flags y mask.
4. **Operadores aritméticos:** se aplican los operadores aritméticos +, -, *, /, y % para realizar operaciones matemáticas básicas.
5. **Operadores de incremento y decremento:** se demuestra el uso de los operadores ++ en sus formas postfix y prefix para incrementar la variable count.

6. **Operadores bitwise:** los operadores `&`, `|`, `^`, `<<`, y `>>` se utilizan para manipular bits en la variable `result`.
7. **Expresión condicional:** la expresión condicional `(today == saturday || today == sunday) ? 'W' : 'b'` se usa para determinar el tipo de día basado en la enumeración `days`.